BASES DE DATOS II

Trabajo Práctico 2

Spring Data JPA

Este segundo trabajo practico consistirá en proveer un nuevo servicio de persistencia utilizando Spring Data JPA en lugar de usar Hibernate directamente, como se hizo en el primer trabajo práctico.
Para lograr esto, deberá crearse una nueva implementación de la interfaz ***ToursService*** llamada ***ToursServiceImpl***, tomando como base la implementación anterior, que deberá utilizar repositorios de Spring Data para resolver cualquier acceso a datos requerido.
En este caso se debe crear un repositorio por clase a persistir. Los repositorios implementados con Spring Data no son clases sino interfaces, y deberán extender `CrudRepository` definiendo los argumentos necesarios: tipo de entidad y tipo de id. Por ejemplo:

```
public interface RouteRepository extends CrudRepository<Route, Long>
```

Las consultas se pueden declarar utilizando query methods, los cuales serán implementados automáticamente por Spring Data (por ejemplo: findBy<Propiedad>), o mediante HQL utilizando la annotation `@Query` (notar que, dado que se trata de una interfaz, los métodos no pueden tener implementación).

Las clases de tests, `ToursApplicationTests` y `ToursQuerysTests` validan, al igual que en la etapa previa, distintas implementaciones del servicio para que pueda probar libremente estas.

Para configurar SpringData se utiliza la clase `SpringDataConfiguration`, que deberán editar con sus credenciales de acceso a la BD.

La infraestructura del proyecto provista se encuentra en el mismo repositorio del TP previo, en la rama *tp2-springdata*:

En concreto, se han modificado: la clase de configuración `AppConfig`, se ha reemplazado `HibernateConfiguration` con la clase de configuración `SpringDataConfiguration` y se han actualizado las dependencias necesarias en `pom.xml`; DBInitializer y por supuesto, los archivos de Testing.

Descripción del dominio

Se quiere generar una app de viajes turísticos para un emprendimiento local.

Una vez que una persona se ha registrado en el sistema de Turismo, puede elegir alguno de los varios recorridos diarios que la empresa ofrece y comprarlo.

Cada recorrido (Route) tiene su nombre, que es representativo del tramo que visita, el valor del mismo, el total en km del recorrido y pueden visualizarse cada una de las paradas (Stop) que componen el recorrido. Todos los recorridos se realizan una vez por día. Además, la aplicación registra comentarios y valoraciones (estrellas) de otros usuarios que han realizado el mismo recorrido.

Dada la cantidad de personas que solicitan los recorridos, la empresa ha tenido que poner un tope máximo de personas por recorrido. Esa cantidad no es general sino que se define para cada uno y se deben vender solo mientras el cupo lo permita.

Adicionalmente, se conoce una serie de choferes y guías turísticos que participan del recorrido.

A modo de ejemplo, podemos pensar en el recorrido "Tour tradicional de Bs. As." que cuenta con paradas en distintos lugares emblemáticos de la ciudad de Buenos Aires como Caminito en La Boca, San Telmo, Obelisco, lago de Palermo y Recoleta; o en "Tour del Delta" que incluye a San Isidro, el Delta y también al Obelisco.

Una vez que se ha registrado la compra de un usuario para un recorrido determinado (Purchase), se le habilita la posibilidad de agregar comentarios y puntaje al recorrido realizado (Review) de forma tal que luego puedan usarlo en promociones del emprendimiento y poder captar mayor público interesado en los mismos.

Se ha pensado que los usuarios (User) se identifiquen en la aplicación mediante usuario y contraseña, y otros datos que pueden resultar útiles para posteriores ofertas de tours que la empresa realice. Por eso se les solicitan datos personales y de contacto como el nombre, un número de celular, un email, edad y fecha de nacimiento.

Los choferes (DriverUser) y guías turísticos (TourGuideUser) también son usuarios del sistema, por lo que se registran los mismos datos que los usuarios, pero además cuentan con una serie de antecedentes que la empresa quiere mantener. El chofer, de trabajos anteriores; el guía turístico, sobre su formación en turismo. Cabe mencionar que los choferes y los guías no están afectados a un recorrido en particular, sino que pueden participar de varios según la demanda y necesidad de la empresa.

Durante todo el recorrido se ofrecen diversos servicios (Service) que un usuario puede contratar. De los servicios se conoce su nombre, su valor, una descripción y el proveedor que lo brinda. Por ejemplo, la confección de algún recuerdo (un mate pintado artesanalmente, una remera o una foto en un lugar determinado).

De cada proveedor (Supplier) se conocen los servicios que ofrece, la razón social y un número de habilitación provincial.

Cada ítem de servicio aparece en la compra junto con el recorrido y la cantidad solicitada del mismo (ItemService).

La compra final contendrá el recorrido elegido y todos los servicios solicitados a lo largo del mismo para que el usuario pueda tener control de lo abonado. Además, sobre cada compra se registra un código interno y la fecha.

Tip *Este trabajo práctico toma como base el proyecto desarrollado en la Práctica 1. El objetivo es migrar la capa de acceso a datos de Hibernate directo a Spring Data JPA, explorando sus ventajas y nuevas capacidades. El mapeo de entidades ya implementado no debe modificarse. Se proporcionarán tests para verificar cada implementación.*

Diagrama de Clases del Modelo

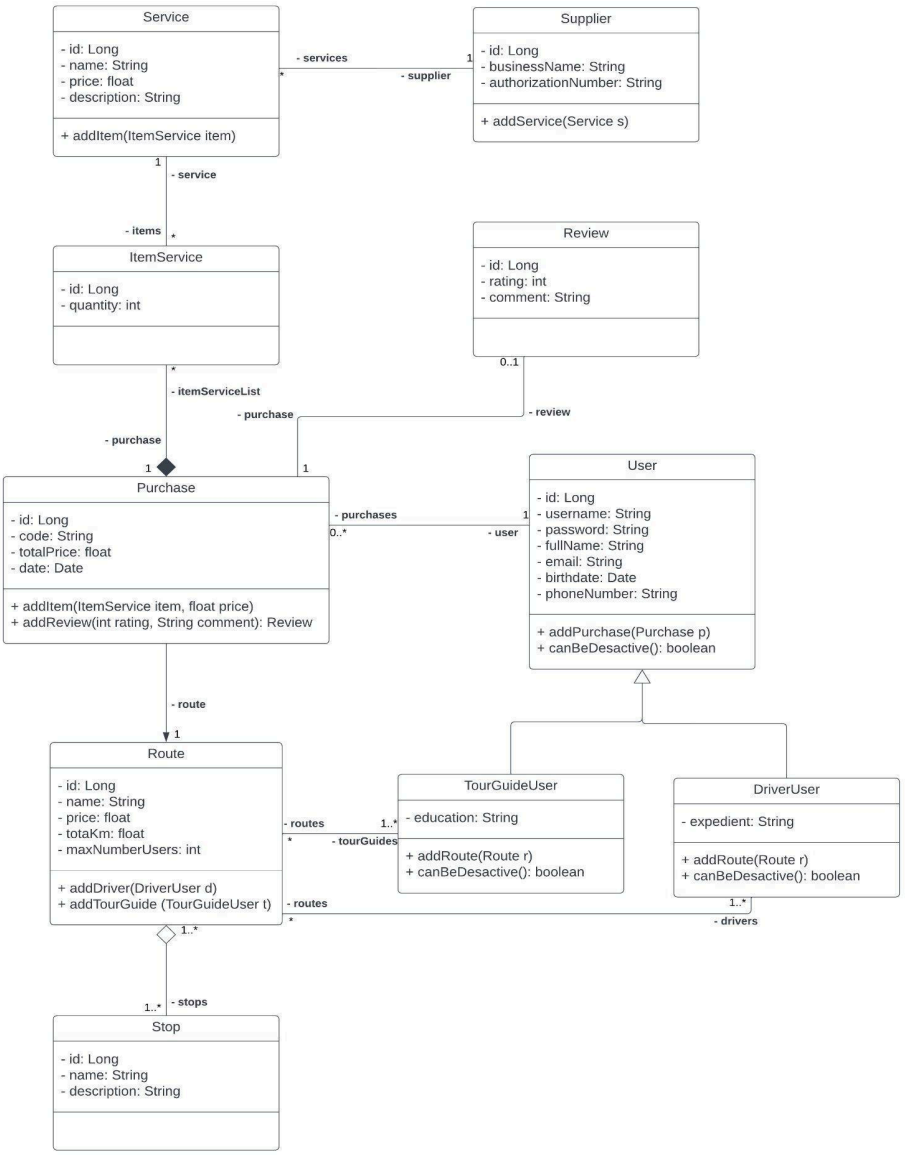

Figura 1 - Diagrama de clases del sistema de gestión de tours. El mapeo de estas entidades ya fue implementado en la Práctica 1.

Entidad	Descripción	Relaciones principales
Service	Servicio ofrecido con nombre y precio	Pertenece a un Supplier; referenciado por ItemService
Supplier	Proveedor que ofrece servicios	Tiene muchos Service

Entidad	Descripción	Relaciones principales
ItemService	Item de servicio en una compra (cantidad)	Referencia Service; pertenece a Purchase
Purchase	Compra de un usuario sobre una ruta	Tiene ItemService, User, Route y Review (0..1)
Review	Reseña con rating y comentario	Asociada a una Purchase (0..1)
User	Usuario base del sistema	Subclasificado en DriverUser y TourGuideUser
DriverUser	Conductor (subclase de User)	Asignado a una o más Route como driver
TourGuideUser	Guia turistico (subclase de User)	Asignado a una o más Route como tourGuide
Route	Ruta con nombre, precio y capacidad	Tiene Stop (1..*), drivers y tourGuides
Stop	Parada de una ruta	Composición con Route (1..*)

Sección 1 - Introducción a Spring Data JPA

Esta sección introduce el framework Spring Data JPA, su lugar dentro del ecosistema Spring y su relación con la infraestructura de persistencia construida en la Práctica 1. El objetivo es entender qué agrega este framework, cómo lo hace internamente y qué partes del proyecto anterior se conservan sin cambios.

1.1 Spring Data JPA y su lugar en el ecosistema

1. ¿Qué es Spring Data JPA? ¿Qué problema resuelve respecto de usar Hibernate directamente? Describir dos situaciones del proyecto donde Spring Data JPA simplifica código que en la Práctica 1 requería implementación manual.
2. Spring Data JPA no es un ORM sino una capa de abstracción sobre el ORM. Explicar la diferencia: ¿qué hace Spring Data JPA? ¿Qué sigue haciendo Hibernate internamente?
3. La siguiente tabla lista tareas relacionadas con la persistencia. Marcar con una X la columna de la tecnología que resuelve ese problema en la nueva implementación con Spring Data JPA:

Tarea	JDBC	Hibernate	Spring Data JPA
Abrir y cerrar la conexión a la base de datos			
Implementar save(), findById() y deleteById()			
Gestionar el ciclo de vida de las entidades (@Entity)			
Derivar una consulta a partir del nombre del método			
Manejar el pool de conexiones			
Propagar transacciones con @Transactional			
Generar la implementación del repositorio en runtime			
Mapear ResultSet a objetos Java			
Proveer soporte nativo de paginación (Pageable)			

1.2 Configuración del proyecto

Migrar de Hibernate directo a Spring Data JPA implica cambios en la configuración del proyecto, pero no en el mapeo de entidades ni en la lógica de negocio. En la Práctica 1 la configuración de Hibernate se realizaba mediante una clase Java de configuración. En esta práctica se utilizará el archivo application.properties de Spring Boot, que centraliza toda esa configuración de forma declarativa.

4. Listar los cambios que deben realizarse en el proyecto de la Práctica 1 para incorporar Spring Data JPA.
5. En la Práctica 1 se configuraba Hibernate mediante una clase de configuración Java para obtener la SessionFactory. ¿Cómo se reemplaza esta configuración usando application.properties en Spring Boot?
6. Describir las propiedades más relevantes de Spring Data JPA que deben configurarse en application.properties para este proyecto. Incluir al menos: datasource (url, username, password, driver), dialecto de Hibernate, y la propiedad que controla si Hibernate debe crear, actualizar o validar el esquema. ¿Cuál de estos valores conviene usar durante el desarrollo?

> **Tip** *El mapeo de entidades (anotaciones @Entity, @Table, @OneToMany, @ManyToMany, estrategias de herencia, cascadas y FetchType) implementado en la Práctica 1 no debe modificarse. Spring Data JPA lo utiliza tal como está.*

1.3 La interfaz CrudRepository y la jerarquía de repositorios

El punto de entrada a Spring Data JPA es una jerarquía de interfaces de repositorio. Comprender cada nivel de esa jerarquía permite elegir la interfaz adecuada para cada caso de uso.

7. ¿Qué es CrudRepository? ¿De qué interfaz hereda y qué operaciones provee automáticamente?
8. ¿Que agrega cada nivel de la jerarquía respecto del anterior? Describir brevemente las diferencias entre CrudRepository, PagingAndSortingRepository y JpaRepository, indicando que operaciones o capacidades incorpora cada uno
9. Crear las interfaces de repositorio para las entidades del modelo extendiendo CrudRepository. Indicar los parámetros de tipo correctos (entidad e ID) para cada una: PurchaseRepository, RouteRepository, UserRepository, ServiceRepository, SupplierRepository y ReviewRepository.
10. ¿Cómo genera Spring Data JPA la implementación concreta de los repositorios en tiempo de ejecución? ¿Qué rol juega el proxy dinámico de Java en este mecanismo?
11. ¿Que diferencia hay entre save() en Spring Data JPA y session.save() / session.merge() en Hibernate directo? ¿Cómo decide Spring Data JPA si debe hacer un INSERT o un UPDATE?

Sección 2 - Repositorios con Spring Data JPA

En esta sección se reemplazan los repositorios manuales de la Práctica 1 por interfaces Spring Data JPA. El objetivo es implementar la misma funcionalidad con menos código, explorando las distintas formas de definir consultas que ofrece el framework.

2.1 Migración de repositorios

12. Comparar el código de PurchaseRepository de la Práctica 1 (con Session de Hibernate) con el nuevo PurchaseRepository de Spring Data JPA. ¿Cuántas líneas de código se eliminaron? ¿Qué operaciones ya no es necesario implementar manualmente?
13. En la Práctica 1, los repositorios gestionaban internamente la Session. ¿Que recibe ahora un servicio que necesita usar un repositorio Spring Data? ¿Cómo se declara esa dependencia? ¿Dónde queda ahora la lógica de apertura y cierre de sesiones?

> **Atención:** *Al migrar los repositorios, los servicios de la Práctica 1 que invocaban métodos de la Session directamente (session.save, session.get, etc.) deben actualizarse para usar los métodos del nuevo repositorio Spring Data. La lógica de negocio en sí no cambia.*

2.2 Query Methods

Spring Data JPA puede generar consultas automáticamente a partir del nombre del método definido en la interfaz del repositorio. Esta técnica se denomina Query Methods.

14. ¿Cómo funciona la generación de consultas por nombre de método en Spring Data JPA? Describir las palabras clave principales que puede interpretar (findBy, existsBy, countBy, deleteBy) y cómo se combinan con los atributos de la entidad.
15. ¿Cómo se pasan los parametros a un Query Method? ¿Cómo hace Spring Data JPA para asociar cada parámetro del método con la condición correspondiente en la consulta? ¿Qué ocurre si el orden de los parámetros no coincide con el orden de las condiciones en el nombre del método?
16. Investigar y para cada uno de los siguientes requerimientos indicar si es posible resolverlo con un Query Method y escribir la firma del método correspondiente. Si no es posible, explicar por qué:
    a) Buscar todas las Purchase de un usuario dado su username.
    b) Verificar si existe alguna Purchase para una Route dada.
    c) Contar cuantas Review tienen un rating mayor o igual a un valor dado.
    d) Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre.
    e) Buscar un User por su email.
    f) Obtener los top 3 Route con mayor cantidad de Purchase.
17. Investigar cuales son las palabras clave (keywords) disponibles en Spring Data JPA para construir Query Methods. ¿Qué tipos de condiciones, comparaciones y operadores lógicos soporta?

18. ¿Qué limitaciones tienen los Query Methods? Describir al menos tres casos del modelo donde esta técnica resulta insuficiente y es necesario otro enfoque. Describa los otros enfoques posibles para implementar dichas consultas.

> **Tip** *Los Query Methods son útiles para consultas simples sobre uno o dos atributos. Para consultas con JOINs complejos, agrupaciones o subconsultas, usar @Query es más claro y mantenible.*

2.3 Consultas con @Query

La anotación @Query permite escribir consultas directamente sobre el método del repositorio. Antes de utilizarla es importante entender la relación entre los lenguajes de consulta involucrados.

19. En la Práctica 1 se utilizaron consultas HQL (Hibernate Query Language). ¿Que diferencia hay entre HQL y JPQL (Java Persistence Query Language)? ¿Son intercambiables? ¿Cual de los dos acepta @Query por defecto?
20. ¿Que diferencia hay entre una consulta @Query con JPQL y una con nativeQuery = true? ¿Cuándo conviene cada una? Dar un ejemplo concreto del modelo para cada caso.
21. ¿Cómo se pasan parámetros a una consulta @Query? Describir y comparar las dos formas: parámetros posicionales (?1, ?2) y parámetros nombrados (@Param). ¿Cuál es la forma recomendada y por qué?

2.4 Paginación y ordenamiento

Spring Data JPA incorpora soporte nativo para paginación y ordenamiento a través de Pageable y Sort, sin necesidad de modificar las consultas manualmente. Esta funcionalidad es necesaria para algunas de las consultas que se implementaran en la sección siguiente.

22. ¿Qué es la interfaz Pageable? ¿Cómo se construye una instancia de Pageable? ¿Qué información encapsula (número de página, tamaño, ordenamiento)?
23. ¿Que diferencia hay entre los tipos de retorno Page<T> y Slice<T>? ¿Cuándo conviene cada uno? ¿Qué consulta adicional ejecuta Page<T> que Slice<T> no ejecuta?
24. Mostrar como se invocará desde la capa de servicio un método paginado para obtener la segunda página de compras de un usuario, con 10 resultados ordenados por fecha descendente.
25. ¿Cómo se agrega ordenamiento a un Query Method sin usar Pageable? ¿Qué palabra clave se usa en el nombre del método? Mostrar un ejemplo concreto con el modelo.

2.5 Implementación de las consultas del modelo

Las siguientes consultas fueron implementadas en la Práctica 1 usando HQL con Session de Hibernate. El objetivo es reimplementarlas ahora usando repositorios Spring Data JPA, eligiendo para cada una la estrategia más adecuada: Query Method o @Query con JPQL.

26. Para cada consulta, indicar la estrategia elegida (Query Method o @Query) e implementarla en el repositorio correspondiente:

Consulta	Repositorio	Estrategia elegida
getAllPurchasesOfUsername(String username)	PurchaseRepository	
getUserSpendingMoreThan(float amount)	UserRepository	
getTopNSuppliersInPurchases(int n)	SupplierRepository	
getCountOfPurchasesBetweenDates(Date from, Date to)	PurchaseRepository	
getRoutesWithStop(Stop stop)	RouteRepository	
getMaxStopOfRoutes()	RouteRepository	
getRoutesNotSell()	RouteRepository	
getTop3RoutesWithMaxRating()	RouteRepository	
getMostDemandedService()	ServiceRepository	
getTourGuidesWithRating1()	UserRepository	

27. Implementar cada una de las consultas de la tabla anterior en el repositorio correspondiente, utilizando la estrategia indicada. Para las consultas que requieran paginación, utilizar Pageable como parámetro adicional.
28. Comparar la implementación de al menos dos consultas complejas (por ejemplo getTopNSuppliersInPurchases y getTop3RoutesWithMaxRating) entre la Práctica 1 y esta práctica. ¿Qué diferencias hay en términos de cantidad de código, legibilidad y acoplamiento a la infraestructura de persistencia? A partir de esta comparación, ¿que es el código boilerplate y que ventaja concreta representa su reducción para el mantenimiento de un proyecto real?

Sección 3 - Transacciones

En la Práctica 1 se implementó el manejo de transacciones utilizando @Transactional sobre la capa de servicio. En esta sección se profundiza sobre algunos aspectos de la gestión declarativa de transacciones en Spring que no se abordaron anteriormente, y se adapta el servicio a la nueva implementación.

3.1 Aspectos avanzados de @Transactional

29. ¿Cuál es el atributo readOnly = true en @Transactional? ¿Qué optimizaciones activa? ¿En qué métodos del servicio conviene aplicarlo?
30. ¿Cómo maneja Spring el rollback automático? ¿Sobre qué tipos de excepción hace rollback por defecto? ¿Cómo se configura para el rollback? Dar un ejemplo con el modelo.

3.2 Adaptación de la capa de servicio

La capa de servicio implementada en la Práctica 1 ya utiliza @Transactional. Sin embargo, al migrar los repositorios a Spring Data JPA, las llamadas internas al servicio deben actualizarse para usar los nuevos repositorios en lugar de la Session de Hibernate.

31. Revisar y actualizar la capa de servicio de la Práctica 1 para que utilice los repositorios Spring Data JPA. Reemplazar todas las llamadas directas a la Session (session.save, session.get, session.createQuery, etc.) por las operaciones equivalentes de los repositorios. La lógica de negocio y las anotaciones @Transactional ya existentes no deben modificarse salvo que sea necesario por los cambios anteriores.

Sección 4 - DTOs y Proyecciones

Hasta ahora todos los repositorios devuelven entidades completas. En muchos casos de uso real, la aplicación solo necesita un subconjunto de los datos. Para esto Spring Data JPA ofrece proyecciones y DTOs.

4.1 Qué es un DTO y cuando usarlo

32. ¿Que es un DTO (Data Transfer Object)? ¿Cuál es su propósito dentro de la arquitectura de una aplicación? ¿Por qué no siempre conviene devolver una entidad JPA directamente desde la capa de acceso a datos?
33. Describir dos situaciones concretas del modelo donde devolver un DTO sería más adecuado que devolver la entidad completa. Para cada caso indicar qué campos contendría el DTO y por que.
34. ¿Qué riesgos concretos tiene exponer entidades JPA directamente como respuesta de un servicio o endpoint? Mencionar al menos: ciclos de serialización y acoplamiento entre capas.

4.2 Implementación

35. Definir e implementar un DTO que resuma información de las rutas del modelo: nombre de la ruta, cantidad de compras realizadas y el precio promedio de esas compras. Implementar la consulta correspondiente en RouteRepository.

Sección 5 - Borrado Lógico (Soft Delete)

El borrado lógico es una estrategia de persistencia que evita eliminar físicamente registros de la base de datos, marcándolos en cambio como inactivos. Spring Data JPA y Hibernate ofrecen mecanismos para implementarlo de forma transparente para el resto de la aplicación.

5.1 Concepto y estrategias

36. ¿Que es el borrado lógico (soft delete)? Pensar en el modelo de tours: si un usuario decide darse de baja del sistema, ¿tiene sentido eliminar físicamente su registro? ¿Qué ocurriría con todas las compras que ese usuario ha realizado? Describir por qué el borrado lógico resuelve este problema y qué ventaja ofrece frente al borrado físico en este caso concreto.
37. Describir dos estrategias para implementar soft delete en JPA/Hibernate: campo booleano (active/deleted) y campo de fecha (deletedAt). ¿Qué diferencias hay entre ambas en cuanto a consultas y recuperación de datos?

38. ¿Que es la anotación @SQLDelete de JPA? ¿Qué permite hacer? ¿Cómo se combina con @Where para que las consultas ordinarias ignoren los registros borrados? Mostrar un ejemplo de uso sobre la entidad User.

5.2 Implementación en el modelo

39. Implementar soft delete para la entidad User usando @SQLDelete y @Where. La implementación debe:
    g) Agregar el campo necesario a la entidad User (campo y anotaciones).
    h) Redefinir el comportamiento de delete para que marque el registro en lugar de eliminarlo.
    i) Asegurar que todas las consultas ordinarias (findAll, findById, Query Methods) excluyan automáticamente los usuarios borrados.
40. La implementacion actual muestra el metodo canBeDeactivate() en User, DriverUser y TourGuideUser. El objetivo de estos métodos es evitar que se eliminen usuarios que poseen compras o asignaciones y, por tanto queden inconsistencias en la base de datos. Describa como esta nueva implementación afecta dichos métodos.

> **Atención:** *La anotación @Where de JPA se aplica globalmente a todas las consultas que Hibernate genera para esa entidad. Si se necesita consultar usuarios inactivos (por ejemplo, para un panel de administración), es necesario usar SQL nativo o JPQL con una consulta explícita que omita el filtro.*

Al completar esta práctica, el proyecto debe contar con los repositorios Spring Data JPA implementados, las consultas migradas, la capa de servicio actualizada, proyecciones y DTOs funcionando, y soft delete activo para User. Todos los tests provistos deben pasar exitosamente.